インタプリタを作るための Rust 入門

topazus の字句解析・構文解析・評価器を
自分の手で書き上げるのに必要な文法と使い方

Rust 2024 edition(Cargo.toml の設定)/ 2026 年 11 月 11 日作成

目次

1 この文書について 3
2 Cargo とモジュール 4
3 変数と基本の型 6
4 関数と式 8
5 所有権と借用 10
6 構造体と impl 12
7 enum 14
8 パターンマッチ 15
9 Option と Result 17
10 コレクションとイテレータ 19
11 Box と再帰的なデータ型 21
12 トレイトと derive 22
13 ライフタイム入門 24
14 Rc と RefCell:関数とスコープを実装するとき 25
15 テスト 27
16 通しの例:小さな電卓言語 29
17 今の parser.rs のエラーを読む 38
18 よく出るコンパイルエラー 39
19 用語集 40
20 参考資料 41

1 この文書について

CLAUDE.md の「次にやること」の 1〜3 番,つまり

1. tokeniser を直す(位置情報を足す)
2. parser を完成させる
3. AST を直接実行するインタプリタを作り,FizzBuzz を動かす

を自分の手で進めるために必要な Rust の知識を,**使う順番**に並べたものである.Rust の全機能を網羅する文書ではない.ジェネリクスの詳しい話,非同期(async),マクロの自作,unsafe などは,インタプリタを作るまでは要らないので省いた.

各章は次の形で書いてある.

- **文法** … 書き方と意味
- **インタプリタでは** … topazus の字句解析・構文解析・評価のどこで使うか
- 枠で囲んだ **補足** と **つまずきやすい所**

最後の方(第 16 章)に,この文書の文法だけで書いた「小さな電卓言語」の全コードを載せた.実際にコンパイルし,テストが通ることを確認してある.topazus そのものではなく,それより小さい言語なので,topazus のインタプリタを書くときの見本として使ってほしい.

> **インタプリタの全体像(おさらい)**
> ソースコードの文字列は,次の 3 段階で実行される.
>
> ```
> "(+ 1 2)"  ──字句解析──▶  [LParen, Ident("+"), Int(1), Int(2), RParen]
>            ──構文解析──▶  Call { op: "+", args: [Int(1), Int(2)] }   (AST)
>            ──評価────▶  Value::Int(3)
> ```
>
> **トークン**(token)はソースを意味のある最小単位に切ったもの.**AST**(抽象構文木,Abstract Syntax Tree)はプログラムの構造を木の形で表したもの.評価器(evaluator)は AST をたどって値を計算する.この文書の章立ても,おおよそこの順になっている.

2 Cargo とモジュール

2.1 Cargo のコマンド

Cargo は Rust のビルドツール兼パッケージ管理ツールである.topazus のリポジトリの `Cargo.toml` が設定ファイルになっている.

コマンド	すること
`cargo check`	型検査だけ行う.実行ファイルを作らないので速い.書いている途中はこれを何度も回す
`cargo build`	コンパイルして `target/debug/` に実行ファイルを作る
`cargo run`	ビルドしてすぐ実行する.`cargo run -- foo.to` のように `--` の後ろがプログラムへの引数になる
`cargo test`	`#[test]` のついた関数と `tests/` の中身を実行する(第 15 章)
`cargo clippy`	よくない書き方を指摘してくれる.Rust らしい書き方を覚えるのに役立つ
`cargo fmt`	コードを標準の書式に整える

`Cargo.toml` の `edition = "2024"` は,Rust のどの版の文法を使うかの指定である.今の tokeniser.rs で使っている `if let Some(next) = chars.peek() && *next == '>'` という書き方(**let チェーン**)は 2024 edition から使えるようになったもので,古い edition では書けない.ネットで古い記事を読むときは注意する.

2.2 モジュール

Rust では 1 つのファイルが 1 つの**モジュール**(名前のまとまり)になる.`main.rs` に `mod tokeniser;` と書くと,`src/tokeniser.rs` がモジュール `tokeniser` として読み込まれる.

```rust
// src/main.rs
mod tokeniser;          // src/tokeniser.rs を読み込む
mod parser;             // src/parser.rs を読み込む
use crate::tokeniser::lexer;   // crate::tokeniser::lexer を lexer と書けるようにする

fn main() {
    let tokens = lexer(include_str!("./test_code.txt").to_string()).unwrap();
}
```

- `crate` は「このプロジェクトの一番上」を指す.`crate::tokeniser::Token` は「tokeniser モジュールの Token」.
- `use` は長い名前に短い別名をつけるだけで,読み込みはしない(読み込みは `mod` の仕事).
- 別のモジュールから使うものには `pub` をつける.`pub` が無いと,そのモジュールの中からしか見えない.struct のフィールドも別々に `pub` が要る.
- `super::` は 1 つ上のモジュールを指す.テストを書くとき `use super::*;` をよく使う(第 15 章).

> **インタプリタでは**
> ファイルは段階ごとに分けるのが分かりやすい.例えば `tokeniser.rs`(トークンと字句

> 解析),ast.rs(AST の型だけ),parser.rs,interpreter.rs(値と評価),error.rs(エラーと位置情報).AST の型を parser.rs から出しておくと,評価器が parser に依存しなくて済む.

3 変数と基本の型

3.1 let と mut

```
let n = 10;          // 不変.n = 11; はコンパイルエラー
let mut cnt = 0;     // 可変
cnt += 1;
let n: i64 = 10;     // 型を明示する.省略すると推論される
```

topazus の `(let [i64 n] <- 10)` と `(let mut [i64 cnt] <- 0)` は,ほぼこれと同じ意味である.Rust も「基本は不変,必要なときだけ mut」という方針の言語なので,topazus の設計を考えるときの参考になる.

3.2 シャドーイング

同じ名前で `let` し直すと,前の変数を**隠す**(シャドーイング,shadowing)ことができる.書き換えではなく,新しい変数を作っている.

```
let text = "123";              // &str
let text: i64 = text.parse().unwrap();   // 別の変数.型も変えられる
```

字句解析で「文字列として集めてから数値に変換する」ときによく使う.

3.3 数値の型

型	内容
`i8` `i16` `i32` `i64` `i128`	符号つき整数(数字はビット数)
`u8` 〜 `u128`	符号なし整数
`isize` `usize`	ポインタと同じ大きさの整数.**配列の添字や長さは `usize`**
`f32` `f64`	浮動小数点数
`bool`	`true` / `false`
`char`	Unicode の 1 文字(4 バイト).`'a'`,`'∀'`

topazus の計算用の型 `i8`〜`i64` `u8`〜`u64` `f32` `f64` は Rust と同じ名前にしてあるので,インタプリタでは topazus の `i64` の値を Rust の `i64` でそのまま持てばよい.

型の変換は `as` で行う.`let i = n as usize;` のように書く.ただし `as` は範囲外の値を黙って切り詰めるので,失敗を知りたいときは `usize::try_from(n)` を使う(結果は `Result`.第 9 章).

3.4 オーバーフロー

Rust では,デバッグビルドで整数があふれると**パニック**(プログラムの異常停止)し,リリースビルドでは折り返す.インタプリタを書くときは,どちらにも頼らず,メソッドで明示するのがよい.

```
i64::MAX.checked_add(1)    // None(あふれたら None,そうでなければ Some(値))
3i64.checked_add(4)        // Some(7)
i64::MAX.wrapping_add(1)   // -9223372036854775808(折り返す)
```

インタプリタでは

topazus の仕様では「証明できないオーバーフローは実行時チェックで停止」「`+%` は折り返し」である.インタプリタでは `+` を `checked_add`,`+%` を `wrapping_add` で実装すれば,そのまま仕様どおりになる.

4 関数と式

4.1 関数の書き方

```
fn add(lhs: i64, rhs: i64) -> i64 {
    lhs + rhs           // セミコロンなし = この値を返す
}
```

引数は `名前: 型`,返り値は `-> 型` で書く.返り値が無い関数は `-> ()` を省略したもので,`()` は**ユニット型**(値が 1 つだけの型)である.CLAUDE.md の未決定事項にある「`[void]` を `[unit]` と `[never]` に分ける案」の `unit` は,この `()` と同じものである.

4.2 式と文

Rust はほとんどのものが**式**(値を持つもの)である.ブロック `{ ... }` も式で,**最後の式の値がブロックの値**になる.最後にセミコロンをつけると,その値は捨てられて `()` になる.

```
let y = {
    let x = 3;
    x * 2           // ブロックの値は 6
};
let sign = if n < 0 { -1 } else { 1 };   // if も式
```

途中で関数を抜けたいときは `return 値;` と書く.エラーを見つけたときに早めに抜ける用途でよく使う.

> **つまずきやすい所:最後のセミコロン**
>
> ```
> fn double(x: i64) -> i64 {
>     x * 2;      // エラー E0308: 型が合わない.i64 を返すはずが () になっている
> }
> ```
>
> 今の parser.rs の `fn parse_if(...) -> Result<ASTNodes, ParseError> {}` も同じ理由でエラーになっている.中身が空のブロックの値は `()` なので,`Result` を返していないと言われる.書きかけの関数は `todo!()` と書いておくと,コンパイルが通るようになる(実行するとそこで停止する).

4.3 if と loop

```
if a < b { ... } else if a == b { ... } else { ... }

while i < n { i += 1; }

for i in 0..30 { ... }        // 0 から 29 まで.0..=30 なら 30 を含む

let found = loop {            // loop は break で値を返せる
    if cond { break 42; }
};
```

topazus の `(loop 30 [i64 i] -> | ... |)` は,インタプリタでは Rust の `for i in 0..30` で実装できる.

5 所有権と借用

Rust で一番大事で,一番つまずく所である.でも規則そのものは少ない.

5.1 所有権

Rust の値には必ず**所有者**(owner)が 1 つだけいる.所有者の変数がスコープを抜けると,値は自動で解放される.これがあるので Rust には GC(ガベージコレクタ)が無い.

```
let a = String::from("let");
let b = a;              // 所有権が a から b に移る(ムーブ,move)
println!("{a}");        // エラー E0382: a はもう使えない
```

ムーブを避けて複製したいときは `.clone()` を呼ぶ.`i64` や `char` や `bool` のような小さな値は **`Copy`** という性質を持っていて,代入しても勝手にコピーされるのでムーブは起きない.

> **topazus との関係**
> 「値の持ち主は 1 人,渡したらもう使えない」という Rust の所有権は,topazus の数量 1(線形型)ととても近い考え方である.topazus の `(set! a 0 1)` が安全にその場で配列を書き換えられるのは,Rust で `&mut` を持っている人だけが書き換えられるのと同じ理屈である.Rust を書いていると,線形型の感覚が自然と身につく.

5.2 借用

所有権を渡さずに値を使わせたいときは**参照**(reference)を渡す.これを**借用**(borrow)と言う.

```
fn length(s: &String) -> usize { s.len() }      // 読むだけ(共有参照)
fn shout(s: &mut String) { s.push('!'); }       // 書き換える(可変参照)

let mut name = String::from("topazus");
let n = length(&name);
shout(&mut name);
```

借用の規則は 1 つだけである.

同時に「共有参照 `&` をいくつでも」か「可変参照 `&mut` を 1 つだけ」のどちらか.

参照を使っている途中で元の値を書き換えたり,可変参照を 2 つ同時に作ったりするとコンパイルエラー(E0502,E0499)になる.

参照の中身を取り出すときは `*` をつける(**参照外し**).今の tokeniser.rs の `*next == '>'` の `*` がそれである.ただし,メソッド呼び出しや比較の多くは自動で参照外しされるので,`*` が要るのは主に「参照と値を比べる」ときである.

5.3 String と &str

文字列の型は 2 つある.

型	意味	使う場面
`String`	所有している,伸び縮みできる文字列	トークンや AST の中に名前を持たせるとき

`&str`	どこかにある文字列の一部を借りている	関数の引数,文字列リテラル `"let"`

- `String` → `&str`:`&s` や `s.as_str()`.
- `&str` → `String`:`s.to_string()`,`String::from(s)`,`s.into()`.
- 関数の引数は `&String` より `&str` にしておくと,どちらからも呼べて便利である.

```
match name.as_str() {          // String のままでは "let" と比べられない
    "let" => ...,
    "if" => ...,
    _ => ...,
}
```

5.4 スライス

`&[T]` は配列や `Vec` の一部を借りる型(**スライス**)である.`&v[1..3]` のように範囲で作る.関数の引数を `&Vec<T>` ではなく `&[T]` にしておくと扱いやすい.第 8 章のスライスパターンと組み合わせると,「引数がちょうど 2 つ」のような判定がきれいに書ける.

> **つまずきやすい所:とりあえず clone**
> 借用のエラーが消えなくて困ったら,まずは `.clone()` してよい.インタプリタの性能は最初は気にしなくてよく,動くことが先である.慣れてから参照に直せばよい.

6 構造体と impl

6.1 struct

複数の値をまとめた型を**構造体**(struct)と言う.

```
#[derive(Debug, Clone, Copy, PartialEq)]
struct Span {
    line: usize,    // 行(1から)
    col: usize,     // 列(1から)
}

struct Token {
    kind: TokenKind,
    span: Span,
}

let s = Span { line: 1, col: 5 };
println!("{}", s.line);
```

`#[derive(...)]` は「この型に標準の機能を自動でつける」指定で,第 12 章で説明する.

6.2 impl とメソッド

型に関数を生やすには `impl` を使う.第 1 引数が `self` の関数は**メソッド**になり,`値.メソッド()` で呼べる.

```
struct Parser {
    tokens: Vec<Token>,
    pos: usize,
}

impl Parser {
    fn new(tokens: Vec<Token>) -> Self {      // self が無い = 関連関
数.Parser::new(...) で呼ぶ
        Parser { tokens, pos: 0 }             // 変数名とフィールド名が同じなら省略できる
    }
    fn peek(&self) -> Option<&Token> {        // &self:読むだけ
        self.tokens.get(self.pos)
    }
    fn advance(&mut self) {                   // &mut self:書き換える
        self.pos += 1;
    }
}
```

`Self` は「今 impl している型」(ここでは `Parser`)の別名である.

> **インタプリタでは:Vec と pos で管理する**
> 今の parser.rs は `Peekable<Iter<Token>>` を関数から関数へ渡している.これでも書けるが,型が長く,ライフタイム(第 13 章)の問題が出やすい.上のように `Parser` 構造体に `Vec<Token>` と現在位置

> `pos` を持たせ,メソッドで `peek` / `next` / `expect` を作るのが分かりやすい(CLAUDE.md の案).2 つ先を覗く(`self.tokens.get(self.pos + 1)`)のも簡単になる.

7 enum

7.1 データを持つ列挙型

Rust の `enum` は「いくつかの形のうちどれか 1 つ」を表す型で,それぞれの形(**バリアント**,variant)がデータを持てる.トークンや AST や実行時の値は,ほぼ全部 enum で表す.

```
#[derive(Debug, Clone, PartialEq)]
enum TokenKind {
    LParen,              // データなし
    Int(i64),            // タプル風:値を1つ持つ
    Ident(String),
    LeftArrow,
}

enum Expr {
    Int(i64),
    Var(String),
    Let { name: String, value: Box<Expr> },   // 構造体風:名前つきフィールド
    Call { op: String, args: Vec<Expr> },
}

let t = TokenKind::Int(42);
let e = Expr::Let { name: "x".to_string(), value: Box::new(Expr::Int(1)) };
```

`Box<Expr>` については第 11 章で説明する.

> **関連知識:代数的データ型**
> enum のような「A か,B か,C か」の型を**直和型**,struct のような「A と B と C」の型を**直積型**と言い,合わせて**代数的データ型**(ADT)と呼ぶ.CLAUDE.md の未決定事項「struct / enum の構文」は,まさにこれを topazus にどう入れるかという話である.カリー=ハワード対応では,直和型が「または」,直積型が「かつ」に対応する(別冊参照).Rust の enum を使い込むと,topazus の enum の設計にも役立つ.

8 パターンマッチ

enum を使うときは,必ず「どのバリアントか」で場合分けする.それが match である.

8.1 match の基本

```
match token.kind {
    TokenKind::LParen => parse_list(),
    TokenKind::Int(n) => Ok(Expr::Int(n)),          // 中身を n に取り出す
    TokenKind::Ident(name) => Ok(Expr::Var(name)),
    other => Err(format!("{other:?} doesn't belong here")),   // 残り全部
}
```

- 腕(arm)は上から順に試され,最初に合ったものが実行される.
- match も式なので値を返す.すべての腕の型はそろっている必要がある.
- **網羅性**:すべての場合を書かないとコンパイルエラー(E0004)になる.残りをまとめるには _(捨てる)か,変数名(束縛する)を使う.

> **つまずきやすい所:網羅性エラー**
> 今の parser.rs の `impl Display for ParseError` は,ParseError に NotExistenceFunction を足したのに match に腕を足していないので E0004 になっている.これは**良いエラー**である.enum にバリアントを増やしたとき,扱い忘れた場所をコンパイラが全部教えてくれる.なので,enum の match ではなるべく _ を使わず,全部のバリアントを書いておくとよい.

8.2 いろいろなパターン

```
match c {
    '(' | ')' => ...,                       // または
    '0'..='9' => ...,                       // 範囲
    c if c.is_alphabetic() => ...,          // ガード(if 条件)
    _ => ...,
}

match (a % 3, a % 5) {                       // タプルで同時に場合分け
    (0, 0) => "FizzBuzz",
    (0, _) => "Fizz",
    (_, 0) => "Buzz",
    _ => "",
}

match expr {
    Expr::Let { name, value } => ...,       // 構造体風バリアントの取り出し
    Expr::Call { op, .. } => ...,           // .. で残りのフィールドを無視
    _ => ...,
}

match (op, args) {                           // スライスパターン
```

```
    ("+", [Value::Int(a), Value::Int(b)]) => ..., // 引数がちょうど2つで,両方 Int
    ("println", [v]) => ...,                      // 引数がちょうど1つ
    _ => ...,
}
```

タプルのパターンは topazus の FizzBuzz の `(match ((% num 3), (% num 5)) { (0, 0) -> ... })` とそっくりである.topazus の `match` を実装するときは,この Rust の `match` の動きを真似すればよい.

8.3 参照に対するマッチ

`&Token` のような参照に `match` すると,取り出した中身も自動で参照になる.

```
fn describe(t: &TokenKind) {
    match t {
        TokenKind::Int(n) => println!("{n}"),        // n は &i64
        TokenKind::Ident(s) => println!("{s}"),      // s は &String
        _ => {}
    }
}
```

参照から `String` を取り出して自分のものにしたいときは `s.clone()` する.所有している値に `match` すると中身はムーブされ,元の値は使えなくなる.

8.4 if let と while let

1 つのパターンだけ気にするときは `if let` が短い.

```
if let Some(t) = self.peek() { ... }             // Some のときだけ
while let Some(c) = chars.next() { ... }         // None になるまで繰り返す

// let チェーン(2024 edition):パターンと条件を && でつなぐ
if let Some(next) = chars.peek() && *next == '>' { ... }
```

8.5 let else

「パターンに合わなければ関数を抜ける」ときは `let ... else` が便利である.`else` の中は必ず `return` などで抜ける必要がある.

```
let Expr::Var(op) = head else {
    return Err("a list must start with a name".to_string());
};
// ここから先では op が使える
```

構文解析では「次は名前のはず,違ったらエラー」という場面がとても多いので,これを覚えるとコードが平らになる.

8.6 matches!

「このパターンに合うかどうか」だけを `bool` で知りたいときは `matches!` マクロを使う.

```
if matches!(t.kind, TokenKind::RParen) { break; }
```

9 Option と Result

Rust には null も例外も無い.代わりに,「無いかもしれない」と「失敗するかもしれない」を型で表す.

9.1 Option

```
enum Option<T> { Some(T), None }
```

T は**型引数**で,どんな型でも入れられることを表す.Vec::get や Peekable::peek や HashMap::get は,値が無いかもしれないので Option を返す.

9.2 Result

```
enum Result<T, E> { Ok(T), Err(E) }
```

成功したら Ok(値),失敗したら Err(エラー).字句解析・構文解析・評価の関数は,全部 Result を返すことになる.

9.3 ? 演算子

Result を返す関数の中で 式? と書くと,「Err ならそのまま関数から返す,Ok なら中身を取り出す」という意味になる.

```
fn parse_let(&mut self) -> Result<Expr, Error> {
    let name = self.parse_name()?;        // 失敗したらここで return Err(...)
    self.expect(TokenKind::LeftArrow)?;
    let value = self.parse_expr()?;
    Ok(Expr::Let { name, value: Box::new(value) })
}
```

? が無いと match の山になる.インタプリタのコードの読みやすさは ? にかかっている.

> **つまずきやすい所:? のエラー型**
> ? で返すエラーの型は,関数の返り値のエラー型と合っている(または変換できる)必要がある.parse::<i64>() が返すエラーは ParseIntError なので,自分の Error 型を返す関数でそのまま ? すると型が合わない.そういうときは map_err で自分のエラーに変換してから ? する.
>
> ```
> let n = text.parse::<i64>().map_err(|_| Error { span, message: "too
> large".into() })?;
> ```

9.4 よく使うメソッド

メソッド	意味
opt.ok_or(err)	Option を Result にする.None なら Err(err)
opt.ok_or_else(|| ...)	同上.エラーを作るのが重いときはこちら
res.map_err(|e| ...)	Err の中身だけ変換する
x.map(|v| ...)	Some / Ok の中身だけ変換する
opt.cloned()	Option<&T> を Option<T> にする(中身を clone)

`opt.unwrap_or(default)`	`None` のときの代わりの値を指定して取り出す
`x.unwrap()`	中身を取り出す.`None` / `Err` ならパニック
`x.expect("理由")`	`unwrap` と同じだが,パニック時のメッセージを指定できる
`x.is_some()` `x.is_ok()`	`bool` で判定

`|v| ...` は**クロージャ**(その場で作る無名関数)である.`|引数| 式` の形で書く.

> **unwrap はどこで使ってよいか**
>
> `unwrap` は「絶対に失敗しない」と分かっている所と,テストの中だけで使う.ユーザーの書いた `.to` ファイルが原因で失敗しうる所(字句解析・構文解析・評価のほぼ全部)では使わず,`?` でエラーを返す.topazus は「エラー文で場所と原因を必ず示す」方針なので,パニックで落ちるのはよくない.

9.5 Box<dyn Error>

今の tokeniser.rs は `Result<Vec<Token>, Box<dyn std::error::Error>>` を返している.`Box<dyn Error>` は「`Error` トレイトを実装した型なら何でも入る箱」で,どんなエラーでも `?` で返せて手軽である.ただし,中身が何のエラーか分からなくなり,位置情報も持たせにくい.インタプリタでは,自分のエラー型(位置と文を持つ struct)を作るのがおすすめである(第 12 章).

10 コレクションとイテレータ

10.1 Vec

`Vec<T>` は伸び縮みする配列である.

```
let mut tokens: Vec<Token> = Vec::new();   // vec![] でも同じ
tokens.push(t);                            // 末尾に追加
tokens.len();                              // 長さ(usize)
tokens[i];                                 // 添字アクセス.範囲外ならパニック
tokens.get(i);                             // Option<&Token>.範囲外なら None
tokens.last();                             // 最後の要素.Option<&Token>
```

`[]` は範囲外でパニックし,`get` は `None` を返す.この区別は topazus の「範囲が証明できれば `[]` 相当,できなければ `get?`」という設計と同じ発想である.

10.2 HashMap

`HashMap<K, V>` はキーから値を引く表で,インタプリタでは**環境**(変数名 → 値の表)に使う.

```
use std::collections::HashMap;

let mut env: HashMap<String, Value> = HashMap::new();
env.insert("x".to_string(), Value::Int(10));
match env.get("x") {                // Option<&Value>
    Some(v) => ...,
    None => ...,                    // 未定義の変数
}
```

10.3 文字を 1 つずつ読む

字句解析では,文字列を 1 文字ずつ見ていく.方法は 2 つある.

```
// 方法1:イテレータ + peekable(今の tokeniser.rs)
let mut chars = src.chars().peekable();
while let Some(c) = chars.next() {
    if let Some(&next) = chars.peek() { ... }   // 次の文字を消費せずに覗く
}

// 方法2:Vec<char> + 位置(添字)
let chars: Vec<char> = src.chars().collect();
let mut pos = 0;
while pos < chars.len() {
    let c = chars[pos];
    let next = chars.get(pos + 1);              // 2つ先も覗ける
    ...
}
```

方法 2 は,`(< x -1)` の `<-` 問題のように「2 文字以上先を見て判断したい」ときや,行と列を数えるときに扱いやすい.

> **つまずきやすい所:文字列の添字**
> `String` に `s[3]` のような添字アクセスはできない.Rust の文字列は UTF-8 で,1 文字のバイト数がまちまち(`a` は 1 バイト,`∀` は 3 バイト)だからである.文字単位で扱いたいときは,上の方法 2 のように `Vec<char>` にしてしまうのが簡単である.

`char` の便利なメソッド:`is_whitespace()`,`is_ascii_digit()`,`is_alphabetic()`,`is_alphanumeric()`,`to_digit(10)`.

10.4 イテレータ

イテレータは「要素を 1 つずつ取り出せるもの」である.`for` 文はイテレータを回している.

```
for t in &tokens { ... }            // 借りて回す(t は &Token)
for t in tokens { ... }             // 所有権ごと回す(以後 tokens は使えない)

let kinds: Vec<TokenKind> = tokens.iter().map(|t| t.kind.clone()).collect();
let text: String = chars[start..pos].iter().collect();   // char の列 → String
```

- `.iter()` は参照を,`.into_iter()` は値そのものを取り出す.
- `.map(f)` は各要素を変換,`.filter(f)` は絞り込み,`.collect()` で `Vec` や `String` にまとめる.

`collect` は `Result` にも集められる.「全部成功なら `Ok(Vec)`,1 つでも失敗したら最初の `Err`」になるので,関数呼び出しの引数を全部評価するときに便利である.

```
let values = args.iter()
    .map(|a| self.eval(a))
    .collect::<Result<Vec<Value>, String>>()?;
```

`::<...>` は**ターボフィッシュ**と呼ばれる書き方で,型推論できないときに型を指定する.

11 Box と再帰的なデータ型

AST は木なので,Expr の中に Expr が入る.しかし次のようには書けない.

```
enum Expr {
    Int(i64),
    Neg(Expr),     // エラー E0072: 大きさが無限になる
}
```

Rust は型の大きさをコンパイル時に知る必要があるが,Expr の中に Expr を直接入れると大きさが決まらない.そこで Box を使う.

```
enum Expr {
    Int(i64),
    Neg(Box<Expr>),                // Box はヒープへのポインタなので大きさが決まる
    Call { op: String, args: Vec<Expr> },   // Vec も中身はヒープにあるので Box は要らない
}

let e = Expr::Neg(Box::new(Expr::Int(1)));
```

Box<T> は「ヒープに置いた T を 1 人で所有するポインタ」である.*b で中身にアクセスでき,メソッドは b.method() でそのまま呼べる.

> **Vec<Box<Expr>> は要らない**
> 今の parser.rs には body: Vec<Box<ASTNodes>> があるが,Vec は中身をもともとヒープに置くので,Vec<ASTNodes> で十分である.Box が必要なのは,Expr を Vec や Option を通さず直接入れるところだけである.

12 トレイトと derive

12.1 トレイト

トレイト(trait)は「この型はこういう機能を持つ」という約束である.Java のインターフェース,Haskell の型クラスに近い.

```
trait Describe {
    fn describe(&self) -> String;
}

impl Describe for Token {
    fn describe(&self) -> String { format!("{:?} at {}:{}", self.kind,
self.span.line, self.span.col) }
}
```

12.2 derive で自動実装する

よく使うトレイトは `#[derive(...)]` で自動で実装できる.

トレイト	できるようになること	インタプリタでの用途
`Debug`	`{:?}` で中身を表示できる	トークン列や AST を表示して確認する.ほぼ全部の型につける
`Clone`	`.clone()` で複製できる	`Token`,`Expr`,`Value`
`Copy`	代入でコピーされる(ムーブしない)	`Span` のような小さな型だけ
`PartialEq`	`==` で比べられる	トークンの比較,テストの `assert_eq!`
`Default`	`T::default()` で初期値を作れる	空の環境など

`Copy` をつけるには,中身が全部 `Copy` である必要がある(`String` を含む型には付けられない).`f64` を含む型は `PartialEq` は付けられるが,`Eq` や `Hash` は付けられない(`NaN != NaN` のため).

12.3 Display:人間向けの表示

`{:?}`(Debug)は開発者向け,`{}`(Display)は利用者向けの表示である.`Display` は自分で実装する.

```
use std::fmt;

impl fmt::Display for Value {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Value::Int(n) => write!(f, "{n}"),
            Value::Bool(b) => write!(f, "{b}"),
            Value::Unit => write!(f, "()"),
        }
    }
}
```

`Display` を実装すると `.to_string()` も使えるようになる.topazus の `(println x)` は,この `Display` を使って実装すればよい.

12.4 自分のエラー型

位置情報つきのエラー型を作り,`Display` を実装しておくと,エラー文を一か所で整えられる.

```
#[derive(Debug)]
struct Error {
    span: Span,
    message: String,
}

impl fmt::Display for Error {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}行{}列: {}", self.span.line, self.span.col, self.message)
    }
}

impl std::error::Error for Error {}   // 中身は空でよい.Box<dyn Error> にも入れられるようになる
```

> **皮肉なエラー文の置き場所**
>
> topazus の個性である皮肉たっぷりのエラー文は,`message` に入れる文字列で表現する.`format!("what on earth is '{other}' supposed to be?")` のように,場所は `span` に,原因と皮肉は `message` に,と分けておくと「皮肉だけど場所と原因は必ず分かる」という約束を守りやすい.

12.5 format! と println!

```
let name = "x";
format!("{name} is undefined")          // 変数を直接埋め込む
format!("{} is undefined", name)        // 位置で埋め込む
format!("{:?}", token)                  // Debug で埋め込む
println!("{v}");                        // 標準出力へ
eprintln!("error: {msg}");              // 標準エラー出力へ
```

名前の後ろの！は**マクロ**の印である.普通の関数と違い,引数の数が自由だったり,コンパイル時に書式を検査したりできる.

13 ライフタイム入門

13.1 ライフタイムとは

参照には「いつまで有効か」という情報がついている.これを**ライフタイム**(lifetime)と言い,`'a` のように書く.ほとんどの場合は省略でき,コンパイラが推論してくれる.省略できないのは,「返り値の参照が,どの引数から借りたものか」が分からないときである.

```
fn longest<'a>(a: &'a str, b: &'a str) -> &'a str {
    if a.len() >= b.len() { a } else { b }
}
```

これは「返り値は,`a` と `b` の両方が生きている間だけ使える」という意味である.

13.2 インタプリタで出てくる場面

構造体に参照を持たせると,ライフタイムを書く必要がある.

```
struct Lexer<'src> {
    src: &'src str,       // ソースコードを借りて持つ
    pos: usize,
}
```

今の parser.rs の `Peekable<Iter<Token>>` も,本当は `Peekable<Iter<'a, Token>>` という,`Vec<Token>` を借りている型である.関数の引数に書くだけなら省略できるが,構造体に入れたり,返り値で返したりしようとするとライフタイムを書く必要が出てくる.

> **ライフタイムを避ける方法**
>
> 最初のインタプリタでは,ライフタイムで悩まないように作るのがおすすめである.
>
> - 構造体には参照ではなく**所有した値**を持たせる(`&str` ではなく `String`,`Iter` ではなく `Vec` と `pos`).
> - 迷ったら `clone` する.
> - AST の名前は `String` で持つ.
>
> この文書の第 16 章のコードには,ライフタイムの注釈が 1 つも出てこない.

14 Rc と RefCell:関数とスコープを実装するとき

第 16 章の電卓言語は変数表が 1 つだけだが,topazus には関数があるので,**スコープ**(変数の見える範囲)を実装する必要がある.ここで `Rc` と `RefCell` が出てくる.FizzBuzz を動かすだけなら後回しでもよい.

14.1 環境の親子

関数を呼ぶと,引数の入った新しい環境ができる.その中で見つからない変数は,外側の環境から探す.

```
グローバル環境   { fizzbuzz: <関数>, main: <関数> }
      ▲ parent
関数呼び出しの環境   { num: 3 }
```

14.2 Rc:所有者を複数にする

関数の値は「自分が定義された環境」を覚えている必要がある(これを**クロージャ**と言う).すると,1 つの環境を「子の環境」と「関数の値」の両方が持つことになり,所有者が 1 人という規則に反する.

`Rc<T>`(参照カウント,reference counting)を使うと,所有者を複数にできる.`Rc::clone(&x)` はデータを複製せず,カウントを 1 増やすだけである.最後の所有者がいなくなるとデータが解放される.

> **topazus との関係**
> topazus のメモリ管理の方針「参照カウント+共有されていなければその場で書き換え」は,Rust の `Rc` とほぼ同じ仕組みである.`Rc::strong_count(&x)` でカウントを見られ,`Rc::get_mut` や `Rc::make_mut` は「カウントが 1 ならその場で書き換え,そうでなければ複製」をする.topazus のランタイムを考えるときに,`Rc` の実装はよい手本になる.

14.3 RefCell:共有したまま書き換える

`Rc` の中身は共有されているので,普通は書き換えられない(共有参照しか作れない).`RefCell<T>` に入れると,借用の規則を**実行時**に検査する代わりに,共有したまま書き換えられるようになる.

```
use std::cell::RefCell;
use std::collections::HashMap;
use std::rc::Rc;

#[derive(Debug, Default)]
struct Env {
    vars: HashMap<String, Value>,
    parent: Option<Rc<RefCell<Env>>>,
}

impl Env {
    fn new_child(parent: Rc<RefCell<Env>>) -> Rc<RefCell<Env>> {
        Rc::new(RefCell::new(Env { vars: HashMap::new(), parent: Some(parent) }))
    }
    fn define(&mut self, name: &str, v: Value) {
        self.vars.insert(name.to_string(), v);
```

```
    }
    fn lookup(&self, name: &str) -> Option<Value> {
        match self.vars.get(name) {
            Some(v) => Some(v.clone()),
            None => self.parent.as_ref()?.borrow().lookup(name),   // 親を探す
        }
    }
}

let global = Rc::new(RefCell::new(Env::default()));
global.borrow_mut().define("x", Value::Int(1));      // borrow_mut で書き換え
let local = Env::new_child(Rc::clone(&global));
local.borrow().lookup("x");                           // Some(Int(1))
```

- `.borrow()` は共有参照,`.borrow_mut()` は可変参照を取り出す.
- 同時に `borrow_mut` を 2 つ取ろうとすると,コンパイルエラーではなく**実行時にパニック**する.`.borrow_mut()` の結果を変数に入れて長く持ち続けないのがコツである.

> **つまずきやすい所:循環参照**
> 関数の値が環境を持ち,その環境に関数の値自身が入っていると,`Rc` が輪になってカウントが 0 にならず,メモリが解放されない(**メモリリーク**).インタプリタが小さいうちは問題にならないので気にしなくてよいが,topazus 本体のメモリ管理を考えるときには,参照カウントのこの弱点を覚えておく必要がある(GC がある言語では起きない).

15 テスト

15.1 単体テスト

```
#[cfg(test)]                // テストのときだけコンパイルする
mod tests {
    use super::*;           // 外側のモジュールのものを全部使う

    #[test]
    fn tokenise_left_arrow() {
        let kinds: Vec<TokenKind> =
            tokenise("(let x <- 1)").unwrap().into_iter().map(|t|
t.kind).collect();
        assert_eq!(kinds, vec![
            TokenKind::LParen, TokenKind::Ident("let".into()),
TokenKind::Ident("x".into()),
            TokenKind::LeftArrow, TokenKind::Int(1), TokenKind::RParen,
        ]);
    }

    #[test]
    fn error_has_position() {
        let err = tokenise("(+ 1\n  &)").unwrap_err();
        assert_eq!(err.span, Span { line: 2, col: 3 });
    }
}
```

- `assert_eq!(a, b)` は `a == b` でなければ失敗する.失敗すると両方の値が表示される(なので `Debug` と `PartialEq` が要る).
- `assert!(条件)` は条件が `false` なら失敗.
- `cargo test tokenise` のように名前の一部を渡すと,そのテストだけ実行できる.

tokeniser の既知の問題(`i64` が分かれる,`==` を読めない,`(< x -1)` など)は,直す前に**失敗するテスト**として書いておくと,直ったことが確認しやすい.

15.2 「コード」と「期待する出力」のペア

CLAUDE.md の方針「`tests/` に『コード』と『期待する出力』のペアを置く」は,次のように実装できる.

```
tests/
  cases/
    fizzbuzz.to         ← topazus のコード
    fizzbuzz.out        ← 期待する出力
  run_cases.rs          ← 全部のペアを実行して比べるテスト
```

```rust
// tests/run_cases.rs
use std::fs;
use std::process::Command;

#[test]
fn all_cases() {
    for entry in fs::read_dir("tests/cases").unwrap() {
        let path = entry.unwrap().path();
        if path.extension().and_then(|e| e.to_str()) != Some("to") { continue; }
        let expected = fs::read_to_string(path.with_extension("out")).unwrap();
        let output = Command::new(env!("CARGO_BIN_EXE_original_lang"))   // ビルドした実行ファイル
            .arg(&path)
            .output()
            .unwrap();
        assert_eq!(String::from_utf8_lossy(&output.stdout), expected, "{}", path.display());
    }
}
```

`tests/` の中のファイルは外から実行ファイルを呼ぶ**結合テスト**になる．`env!("CARGO_BIN_EXE_original_lang")` は Cargo が用意してくれる，ビルド済みの実行ファイルのパスである（`original_lang` は Cargo.toml の `name`）．この形にするには，`main` が `.to` ファイルのパスを引数で受け取るようにしておく．

```rust
fn main() {
    let path = std::env::args().nth(1).expect("usage: topazus FILE.to");
    let src = std::fs::read_to_string(&path).expect("cannot read the file");
    ...
}
```

このテストの仕組みは，第 5 段階の「C へのコード生成」で，インタプリタと C 版の結果を比べるときにもそのまま使える．

16 通しの例:小さな電卓言語

ここまでの文法だけで書いた,小さな言語の全コードを載せる.`cargo new` したプロジェクトの `src/main.rs` にそのまま貼れば動く.

できること:整数,`true` / `false`,`+ - * / < ==`,`(let x <- 式)`,`(if 条件 式 式)`,`(println 式)`.エラーには行と列がつく.

```
(let x <- 10)
(let y <- (* x 3))
(println (+ x y))          ; 40
(println (if (< x 5) 1 2)) ; 2
```

topazus より小さく,型の注釈 `[i64 x]` も関数も無い.topazus のインタプリタを書くときは,これを手本に機能を足していけばよい.

16.1 字句解析

```
use std::collections::HashMap;
use std::fmt;

#[derive(Debug, Clone, Copy, PartialEq)]
struct Span {
    line: usize,
    col: usize,
}

#[derive(Debug, Clone, PartialEq)]
enum TokenKind {
    LParen,        // (
    RParen,        // )
    Int(i64),      // 123
    Ident(String), // let, x, +, <
    LeftArrow,     // <-
}

#[derive(Debug, Clone)]
struct Token {
    kind: TokenKind,
    span: Span,
}

#[derive(Debug)]
struct Error {
    span: Span,
    message: String,
}
```

```
impl fmt::Display for Error {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{}行{}列: {}", self.span.line, self.span.col, self.message)
    }
}

fn is_ident_char(c: char) -> bool {
    c.is_alphanumeric() || "+-*/<>=!?_".contains(c)
}

fn tokenise(src: &str) -> Result<Vec<Token>, Error> {
    let chars: Vec<char> = src.chars().collect();
    let mut tokens = Vec::new();
    let mut pos = 0;
    let (mut line, mut col) = (1, 1);

    while pos < chars.len() {
        let c = chars[pos];
        let span = Span { line, col };
        if c == '\n' {
            pos += 1;
            line += 1;
            col = 1;
            continue;
        }
        if c.is_whitespace() {
            pos += 1;
            col += 1;
            continue;
        }
        let start = pos;
        let kind = match c {
            '(' => {
                pos += 1;
                TokenKind::LParen
            }
            ')' => {
                pos += 1;
                TokenKind::RParen
            }
            '<' if chars.get(pos + 1) == Some(&'-') => {
                pos += 2;
                TokenKind::LeftArrow
            }
            c if c.is_ascii_digit() => {
                while pos < chars.len() && chars[pos].is_ascii_digit() {
```

```
                    pos += 1;
                }
                let text: String = chars[start..pos].iter().collect();
                let n = text.parse::<i64>().map_err(|_| Error {
                    span,
                    message: format!("{text} is far too large, I'm afraid"),
                })?;
                TokenKind::Int(n)
            }
            c if is_ident_char(c) => {
                while pos < chars.len() && is_ident_char(chars[pos]) {
                    pos += 1;
                }
                TokenKind::Ident(chars[start..pos].iter().collect())
            }
            other => {
                return Err(Error {
                    span,
                    message: format!("what on earth is '{other}' supposed to be?"),
                });
            }
        };
        col += pos - start;
        tokens.push(Token { kind, span });
    }
    Ok(tokens)
}
```

読みどころ

- `Vec<char>` と `pos` で読む(第 10 章の方法 2).`chars.get(pos + 1)` で次の文字を覗く.
- 識別子に数字や `-?!` を許している(`is_ident_char`).そのため `i64` も `add-comm` も 1 つの名前になる.`+` や `<` も名前として扱い,評価器で意味をつける.Lisp 系の言語ではよくあるやり方で,トークンの種類が少なくて済む.
- 知らない文字は黙って無視せず,位置つきのエラーにする.
- `match` の各腕が「何文字進めたか」を `pos` に反映し,最後に `col += pos - start` で列を進める.
- このコードには,今の tokeniser.rs と同じく `(< x -1)` の問題が残っている(`<-` と読む).topazus では,例えば「`<-` の後ろに空白が必要」などの規則を決める必要がある.

16.2 構文解析

```
#[derive(Debug, Clone, PartialEq)]
enum Expr {
    Int(i64),
    Var(String),
    Let { name: String, value: Box<Expr> },
    Call { op: String, args: Vec<Expr> },
```

```
}

struct Parser {
    tokens: Vec<Token>,
    pos: usize,
}

impl Parser {
    fn new(tokens: Vec<Token>) -> Self {
        Parser { tokens, pos: 0 }
    }

    fn peek(&self) -> Option<&Token> {
        self.tokens.get(self.pos)
    }

    fn next(&mut self) -> Option<&Token> {
        let t = self.tokens.get(self.pos);
        self.pos += 1;
        t
    }

    fn last_span(&self) -> Span {
        self.tokens.last().map(|t| t.span).unwrap_or(Span { line: 1, col: 1 })
    }

    fn expect(&mut self, want: TokenKind) -> Result<(), Error> {
        let eof = self.last_span();
        match self.next() {
            Some(t) if t.kind == want => Ok(()),
            Some(t) => Err(Error {
                span: t.span,
                message: format!("expected {:?}, but got {:?}", want, t.kind),
            }),
            None => Err(Error { span: eof, message: "the file ended rather
abruptly".into() }),
        }
    }

    fn parse_program(&mut self) -> Result<Vec<Expr>, Error> {
        let mut exprs = Vec::new();
        while self.peek().is_some() {
            exprs.push(self.parse_expr()?);
        }
        Ok(exprs)
    }
```

```
    fn parse_expr(&mut self) -> Result<Expr, Error> {
        let eof = self.last_span();
        let token = self.next().cloned().ok_or(Error {
            span: eof,
            message: "the file ended rather abruptly".into(),
        })?;
        match token.kind {
            TokenKind::Int(n) => Ok(Expr::Int(n)),
            TokenKind::Ident(name) => Ok(Expr::Var(name)),
            TokenKind::LParen => self.parse_list(),
            other => Err(Error { span: token.span, message: format!("{other:?}
doesn't belong here") }),
        }
    }

    // '(' を読んだ直後から呼ばれる
    fn parse_list(&mut self) -> Result<Expr, Error> {
        let head = self.parse_expr()?;
        let Expr::Var(op) = head else {
            let span = self.tokens[self.pos - 1].span;
            return Err(Error { span, message: "a list must start with a
name".into() });
        };
        if op == "let" {
            let name = match self.parse_expr()? {
                Expr::Var(name) => name,
                _ => {
                    let span = self.tokens[self.pos - 1].span;
                    return Err(Error { span, message: "let needs a name".into() });
                }
            };
            self.expect(TokenKind::LeftArrow)?;
            let value = self.parse_expr()?;
            self.expect(TokenKind::RParen)?;
            return Ok(Expr::Let { name, value: Box::new(value) });
        }
        let mut args = Vec::new();
        while let Some(t) = self.peek() {
            if t.kind == TokenKind::RParen {
                break;
            }
            args.push(self.parse_expr()?);
        }
        self.expect(TokenKind::RParen)?;
        Ok(Expr::Call { op, args })
```

```
    }
}
```

読みどころ

- **再帰下降構文解析**(recursive descent parsing):`parse_expr` が `parse_list` を呼び,`parse_list` がまた `parse_expr` を呼ぶ.s 式は入れ子の構造がそのまま再帰になるので,この方法と相性がとてもよい.
- `self.next().cloned()`:`next` は `Option<&Token>` を返す.参照のままだと `self` を借りっぱなしになり,その後 `self.parse_list()`(`&mut self`)を呼べない(E0502).`cloned()` で自分のものにして借用を終わらせている.
- `let Expr::Var(op) = head else { ... }`:第 8 章の `let else`.
- `let` だけ特別扱いし,それ以外は全部 `Call` にする.`if` も `Call { op: "if", .. }` になり,評価器で特別扱いする.topazus でも `if` / `match` / `loop` のような構文は,構文解析で専用の AST にするのがよい(第 4 段階の型検査がしやすくなる).

16.3 評価

```
#[derive(Debug, Clone, PartialEq)]
enum Value {
    Int(i64),
    Bool(bool),
    Unit,
}

impl fmt::Display for Value {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            Value::Int(n) => write!(f, "{n}"),
            Value::Bool(b) => write!(f, "{b}"),
            Value::Unit => write!(f, "()"),
        }
    }
}

struct Interpreter {
    env: HashMap<String, Value>,
}

impl Interpreter {
    fn new() -> Self {
        Interpreter { env: HashMap::new() }
    }

    fn eval(&mut self, expr: &Expr) -> Result<Value, String> {
        match expr {
```

```
            Expr::Int(n) => Ok(Value::Int(*n)),
            Expr::Var(name) => match name.as_str() {
                "true" => Ok(Value::Bool(true)),
                "false" => Ok(Value::Bool(false)),
                _ => self
                    .env
                    .get(name)
                    .cloned()
                    .ok_or_else(|| format!("{name}? never heard of it")),
            },
            Expr::Let { name, value } => {
                let v = self.eval(value)?;
                self.env.insert(name.clone(), v);
                Ok(Value::Unit)
            }
            Expr::Call { op, args } if op == "if" => {
                let [cond, then, els] = args.as_slice() else {
                    return Err("if wants exactly three things".to_string());
                };
                match self.eval(cond)? {
                    Value::Bool(true) => self.eval(then),
                    Value::Bool(false) => self.eval(els),
                    other => Err(format!("{other} is not a bool, however hard you
wish")),
                }
            }
            Expr::Call { op, args } => {
                let values = args
                    .iter()
                    .map(|a| self.eval(a))
                    .collect::<Result<Vec<Value>, String>>()?;
                apply(op, &values)
            }
        }
    }
}

fn apply(op: &str, args: &[Value]) -> Result<Value, String> {
    match (op, args) {
        ("println", [v]) => {
            println!("{v}");
            Ok(Value::Unit)
        }
        ("+", [Value::Int(a), Value::Int(b)]) => {
            a.checked_add(*b).map(Value::Int).ok_or("overflow.
splendid.".to_string())
```

```
        }
        ("-", [Value::Int(a), Value::Int(b)]) => Ok(Value::Int(a - b)),
        ("*", [Value::Int(a), Value::Int(b)]) => Ok(Value::Int(a * b)),
        ("/", [Value::Int(_), Value::Int(0)]) => Err("dividing by zero.
bold.".to_string()),
        ("/", [Value::Int(a), Value::Int(b)]) => Ok(Value::Int(a / b)),
        ("<", [Value::Int(a), Value::Int(b)]) => Ok(Value::Bool(a < b)),
        ("==", [a, b]) => Ok(Value::Bool(a == b)),
        _ => Err(format!("cannot apply {op} to {args:?}")),
    }
}

fn run(src: &str) -> Result<(), String> {
    let tokens = tokenise(src).map_err(|e| e.to_string())?;
    let program = Parser::new(tokens).parse_program().map_err(|e| e.to_string())?;
    let mut interp = Interpreter::new();
    for expr in &program {
        interp.eval(expr)?;
    }
    Ok(())
}

fn main() {
    let src = "
(let x <- 10)
(let y <- (* x 3))
(println (+ x y))
(println (if (< x 5) 1 2))
";
    if let Err(msg) = run(src) {
        eprintln!("error: {msg}");
        std::process::exit(1);
    }
}
```

読みどころ

- `eval` は AST を受け取って `Value` を返す,`match` による場合分けそのものである.これが「AST を直接実行するインタプリタ」(**tree-walking interpreter**)の基本形である.
- `if` は引数を**全部評価する前に**特別扱いしている.先に全部評価すると,選ばれなかった方の枝まで実行されてしまう(`(if c (println 1) (println 2))` が両方表示する).この違いを**正格評価**(先に引数を評価する)と**特殊形式**(評価の順番を自分で決める構文)と言う.topazus の `if` `match` `while` `loop` は全部特殊形式である.
- `apply` の `match (op, args)` は第 8 章のスライスパターン.「`+` で,引数が 2 つで,両方 `Int`」を 1 行で書ける.型が合わない場合は最後の `_` に落ちる.

- `+` だけ `checked_add` にしてある.topazus の「オーバーフローは実行時チェックで停止」の小さな見本である.
- 評価器のエラーは `String` にして手を抜いてある.topazus では AST にも `Span` を持たせ,評価時のエラーにも位置をつけるとよい.

16.4 この先 topazus に向けて足すもの

足すもの	使う Rust の知識
`[i64 x]` の型注釈を読む	構文解析で `Lbrackets` を `expect` し,型名を enum に変換する(第 7 章)
`{ then -> a, else -> b }`	`expect` を並べるだけ.`Expr::If { cond, then, els }` の専用 AST にする
`match` と `(0, _)` のパターン	`enum Pattern { Int(i64), Wildcard, Tuple(Vec<Pattern>), Var(String) }` を作り,値とパターンを照合する再帰関数を書く
`(loop 30 [i64 i] -> | ... |)`	`for i in 0..n` で回し,`i` を環境に入れて本体を評価
関数とその呼び出し	`Value::Closure` と環境の親子(第 14 章の `Rc<RefCell<Env>>`)
`(println x)` / `(get [i64])`	`Display`(第 12 章),`std::io::stdin().read_line`
文字列リテラル `"Fizz"`	字句解析で `"` から `"` までを集める.`\n` などのエスケープも処理する

17 今の parser.rs のエラーを読む

`cargo build` すると 17 件のエラーが出る.それぞれ,この文書のどの章の話かを対応させておく.エラーは**上から順に**直すのがコツである.前のエラーが原因で後ろのエラーが出ていることが多い.

エラー	原因	章
`expected pattern, found }`	`parse_params` の最後が `let` だけで終わっている.書きかけ	4
E0214 / E0277 / E0658(`Vec(Type, String)`)	`Vec<(Type, String)>` の書き間違い.`Vec(...)` だと別の意味(関数のような型)に読まれて,関係なさそうなエラーがいくつも出る	10
E0425 `tokens` が見つからない	`parse` の引数名が `tonkens` になっている	—
E0425 `pase_expr` が見つからない	`parse_expr` の綴り間違い	—
E0004 網羅していない	`Display` の `match` に `NotExistenceFunction` の腕が無い	8
E0308 型が合わない(8 件)	中身が `{}` の関数が 6 つ(値が `()` になる),`parse_let` の `Type::Fn => {}` の腕,書きかけの `parse_params`	4

> **表のエラーを直すと出てくるエラー**
>
> Rust のコンパイラは,前のエラーがあるとその先の検査を省くことがある.表の 17 件を直すと,次の 2 件が新しく出てくる(実際に試して確認した).
>
> 1 つ目は `parse` の中の `parse_expr(&mut tokens)` である.`tokens` は `Vec<Token>` で,渡すべきなのは `Peekable` にした `token` の方である.名前が似ていて取り違えやすいので,`let mut iter = tokens.iter().peekable();` のように,はっきり違う名前にするとよい.
>
> 2 つ目は `parse_type` の最後の `Ok(ty)` である.`judge_type` の中の `let ty = match ... { ... };` は `Result<Type, ParseError>` になる.それを `parse_type` で `let ty = judge_type(tokens);` と受け取り,最後に `Ok(ty)` と返すと `Ok(Result<Type, ...>)` という二重の形になってしまう.`let ty = judge_type(tokens)?;` と `?` をつければ中身の `Type` が取り出せる(第 9 章).
>
> また `judge_type` は `peek` しかせず,型名の消費は呼び出し側の `parse_type` が `tokens.next()` で行っている.こういう「誰が消費するか」の約束を関数ごとにコメントで書いておくと,構文解析のバグが減る.

> **どこから直すか**
>
> CLAUDE.md の方針は「1 つの機能を字句解析から実行まで通してから次の機能を足す」である.parser.rs の書きかけの関数を全部埋めるより,中身が無い関数を `todo!()` にしてまずコンパイルを通し,`let` と整数と `+` だけを実行まで通すのがおすすめである.

18 よく出るコンパイルエラー

Rust のコンパイラのエラー文はとても丁寧で,`help:` の行に直し方が書いてあることが多い.`rustc --explain E0382` のようにすると,そのエラーの詳しい説明と例が読める.

番号	意味	よくある直し方
E0382	ムーブした後の値を使った	`.clone()` する,または参照 `&` で渡す
E0502	共有参照を使っている途中で可変参照を作った	参照を使い終わってから書き換える.`.cloned()` で参照を切る
E0499	可変参照を 2 つ同時に作った	片方を使い終わってからもう片方を作る
E0308	型が合わない	`expected` と `found` を見比べる.最後のセミコロン,`?` の付け忘れ
E0004	`match` が網羅していない	足りないバリアントの腕を足す
E0425	名前が見つからない	綴り,`use` の書き忘れ
E0433	モジュールやパスが見つからない	`mod` / `use` / `pub` の確認
E0599	そのメソッドが無い	型を確認する.`Option` の中身に直接メソッドを呼んでいないか
E0106	ライフタイムの指定が要る	参照ではなく所有した値(`String` など)を持たせる(第 13 章)
E0072	再帰的な型の大きさが無限	`Box` で包む(第 11 章)
E0277	トレイトが実装されていない	`#[derive(Debug)]` などを足す.`{:?}` や `==` を使うとき

> **一番よく出る組み合わせ:peek してから next**
>
> ```
> if let Some(t) = self.peek() {      // t は self を借りている
>     self.advance();                 // エラー E0502: self を書き換えようとした
>     use_token(t);
> }
> ```
>
> `peek` が返した参照 `t` を使い終わる前に `self` を書き換えようとするとこうなる.`let t = self.peek().cloned();` で複製する,あるいは `t` から必要な値(種類や Span)を先に取り出してから `advance` する.

19 用語集

用語	意味
所有権(ownership)	値を解放する責任を持つのは 1 人だけ,という Rust の規則
ムーブ(move)	所有権が別の変数に移ること.元の変数は使えなくなる
借用(borrow)	所有権を渡さずに参照 `&` / `&mut` で値を使わせること
ライフタイム(lifetime)	参照が有効な期間.`'a` と書く
トレイト(trait)	型が持つ機能の約束.`Debug`,`Clone`,`Display` など
derive	よく使うトレイトを自動で実装する指定
パニック(panic)	回復できないエラーによるプログラムの停止
クロージャ(closure)	その場で作る無名関数.外の変数を取り込める.`|x| x + 1`
マクロ(macro)	コードを生成するもの.名前の後ろに `!`.`println!`,`vec!`
トークン(token)	ソースを意味のある最小単位に切ったもの
AST	抽象構文木.プログラムの構造を表した木
環境(environment)	変数名から値を引く表.スコープごとに作る
再帰下降構文解析	文法の規則ごとに関数を作り,互いに再帰呼び出しして読む方法
tree-walking interpreter	AST をたどって直接実行するインタプリタ
特殊形式(special form)	引数を先に評価しない構文.`if`,`let` など
Span	ソース上の位置(行・列,または開始と終了のバイト位置)
参照カウント(Rc)	その値を指している所有者の数を数え,0 になったら解放する方式

20 参考資料

- **The Rust Programming Language**(通称 The Book):https://doc.rust-lang.org/book/ ,日本語訳 https://doc.rust-jp.rs/book-ja/ .4 章(所有権),6 章(enum と match),9 章(エラー処理)がこの文書の中心と重なる.
- **Rust by Example**:https://doc.rust-jp.rs/rust-by-example-ja/ .短い例で文法を確認したいとき.
- **標準ライブラリのドキュメント**:https://doc.rust-lang.org/std/ `.Option`,`Vec`,`HashMap`,`char` のページは何度も見ることになる.
- **Crafting Interpreters**(Robert Nystrom):https://craftinginterpreters.com/ .インタプリタの作り方を一から説明した本.前半の tree-walking interpreter の部分が,topazus の第 3 段階とちょうど同じ内容である(Java で書かれているが,考え方はそのまま使える).
- `rustc --explain E番号`:エラーの詳しい説明.